FinGuard

Real-Time Credit Card Fraud Detection

An End-to-End Streaming Data Engineering Project on Databricks

Platform: Databricks (Free Edition)

A complete project reference guide — from data ingestion to real-time fraud alerting and dashboards.

Table of Contents

Contents

Table of Contents 2

1. Overview 3

Figure 1: Batch vs Streaming Processing 3

2. Business Use Case 4

Background 4

⚠ Problem 4

3. Architecture 4

Figure 2: End-to-End Data Processing Architecture 4

◇ Confluent Kafka (Streaming Source) 4

◇ Spark Structured Streaming 5

◇ Auto Loader 5

◇ Lakeflow Spark Declarative Pipelines (SDP) 5

◇ Delta Lake 5

◇ Gmail SMTP + Databricks Secret Scope 5

4. Data Sources 5

4.1 Confluent Kafka — Live Transactions (Streaming) 5

Figure 3: Kafka Architecture 6

4.2 JSON Files — Fraud Watchlist (Streaming) 6

4.3 Neon PostgreSQL — Customer Master Data (Batch) 6

5. Data Pipelines 6

5.i) Source → Bronze Ingestion 6

Bronze Tables 7

5.ii) Bronze → Silver Ingestion 7

Bronze → Silver Mapping & Transformations 7

5.iii) Silver → Gold Ingestion 8

a. Stateless vs. Stateful Transformation 8

b. Watermarks 9

5.iv) Sending Real-Time Email Alerts 11

Alert 1 — High-Value Transaction Alert 11

Alert 2 — Fraud Card Alert 11

6. Dashboard 11

Charts & Visualizations 12

7. Project Details for Resume 12

Project Overview & Duration 12

Skills Used 13

Roles & Responsibilities 13

1. Overview

Streaming is the continuous ingestion, processing, and analysis of real-time events that enables low-latency analytics and immediate decisionmaking. Unlike batch processing — where data is collected over hours or days and then processed on a schedule — streaming moves every single event from source to analytics the moment it is generated, keeping latency down to seconds or even sub-seconds. This makes streaming the right choice for use cases such as real-time fraud detection, live dashboards, IoT monitoring, and stock-price tracking, where waiting for a batch to accumulate is simply not acceptable.

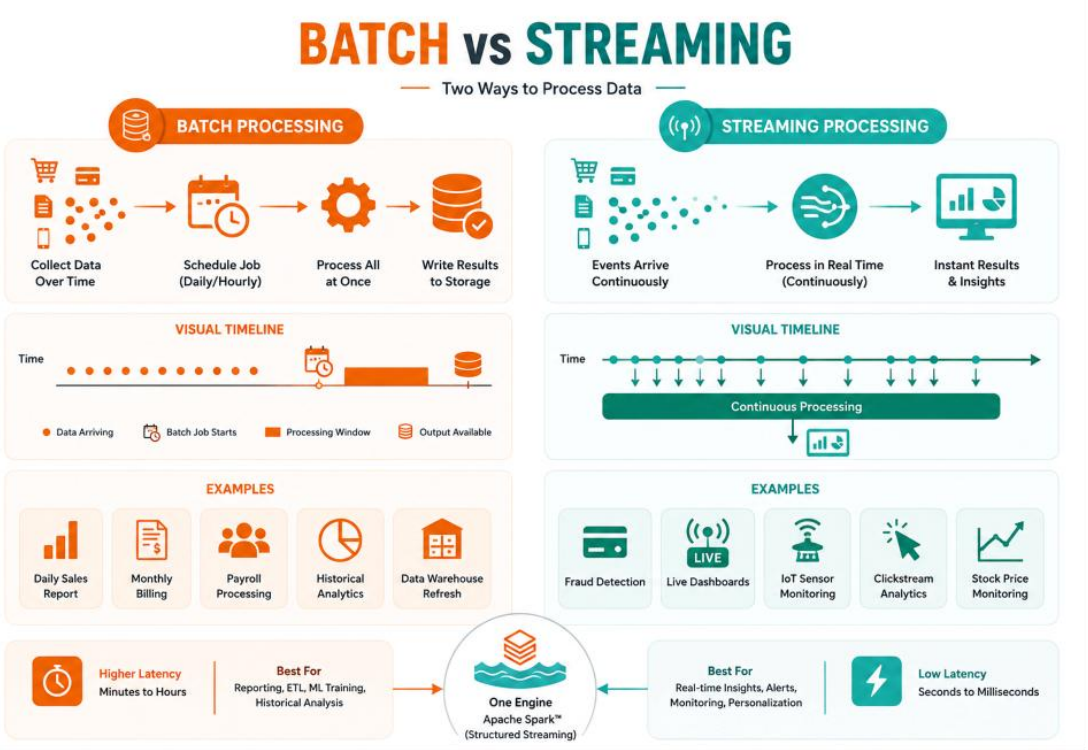


Figure 1: Batch vs Streaming Processing

In **batch processing**, data is collected over a period of time (hourly or daily), then processed together on a schedule, which is ideal for highvolume workloads where some delay is acceptable (e.g., daily sales reports, monthly billing, payroll). In **stream processing**, each event is processed the moment it arrives, giving low latency for immediate decisions (e.g., real-time fraud detection, live dashboards, IoT and stock-price monitoring). Databricks supports both models, and FinGuard uses streaming to detect fraud within seconds.

This project, **FinGuard**, is a complete end-to-end real-time fraud detection system built on Databricks. It ingests live credit card transactions from **Confluent Kafka**, continuously processes fraud watchlist updates arriving as **JSON files**, and enriches both with **customer master data** from a **Neon PostgreSQL** database. The data flows through a standard **Bronze → Silver → Gold medallion architecture** built on **Delta Lake** and governed by **Unity Catalog**. Along the way it demonstrates every important streaming concept — stateless vs. stateful processing, watermarking, stream-static joins, stream-stream joins, and windowed aggregations (tumbling and sliding). Whenever a fraudulent or suspicious transaction is detected, the system sends a **real-time email alert** to the customer, and a **live dashboard** refreshes every minute to give operations teams up-to-the-minute visibility. The result is a production-style blueprint that detects fraud within seconds instead of hours.

---

2. Business Use Case

FinGuard Bank needs to detect fraudulent credit card transactions the moment they happen, rather than discovering them hours later through traditional batch reporting. This project delivers a real-time detection and alerting system to protect both the bank and its customers.

🏦 Background

- FinGuard Bank processes **millions of credit card transactions daily** across ATMs, POS terminals, and online payment gateways.
- The bank receives **continuous fraud intelligence** from internal fraud teams and external security partners.
- **Delayed fraud detection** can result in significant financial losses and poor customer experience.

⚠ Problem

- Fraudulent transactions **must be identified within seconds**.
- Fraud indicators arrive from **multiple sources** — including live transactions, fraud watchlists, and customer master data — with no single system to combine them.
- **Traditional batch processing cannot react quickly enough** to prevent fraudulent activities.

☑ Solution

- **Ingest real-time credit card transactions** from Confluent Kafka using managed streaming pipelines.
- **Continuously process fraud watchlist updates** that arrive as JSON files, ingested in real time with Auto Loader.
- **Enrich transactions** with customer and merchant reference data (customer master data from PostgreSQL).
- Apply **streaming joins and window-based analytics** to detect fraud in real time.
- **Generate real-time email alerts** and **operational dashboards** so the bank can act within seconds.

---

3. Architecture

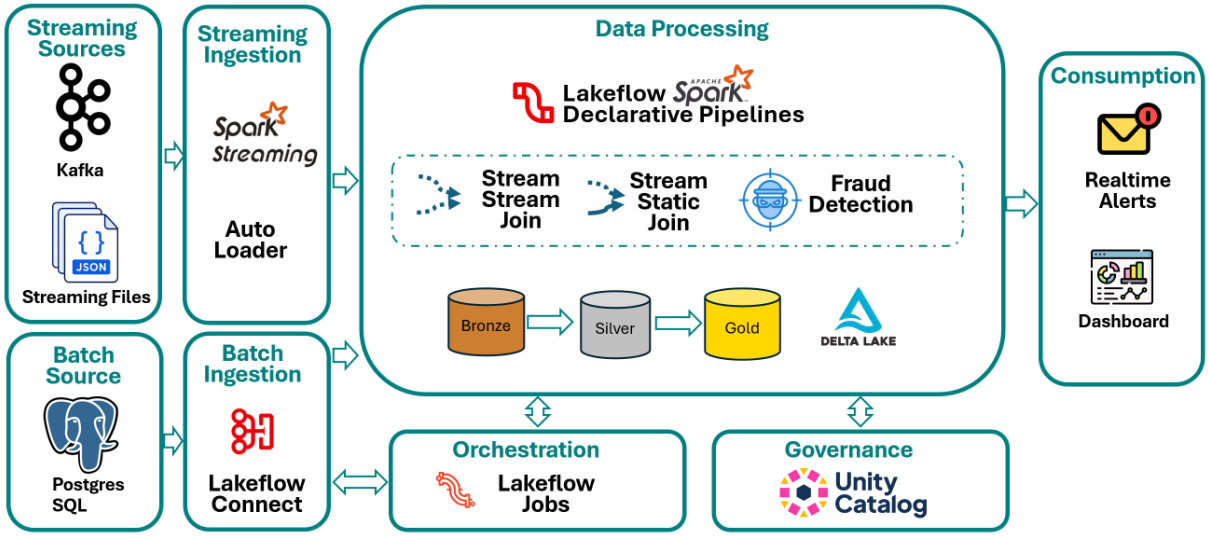


Figure 2: End-to-End Data Processing Architecture

Streaming transactions from **Kafka** (via Spark Streaming) and fraud-watchlist files (via **Auto Loader**), together with batch customer data from PostgreSQL (via **Lakeflow Connect**), flow through **Bronze** → **Silver** → **Gold** layers built with **Lakeflow Declarative Pipelines**. Everything is governed by **Unity Catalog**, orchestrated by **Lakeflow Jobs**, and consumed through **dashboards** and **real-time email alerts**.

The following components make up the FinGuard architecture. For each, we describe **what** we use and **why**. ◇

◇ Confluent Kafka (Streaming Source)

- Acts as the real-time source for live credit card transactions.
- Chosen because Confluent was founded by the original creators of Apache Kafka, making it the most reliable and widely-used managed streaming platform.

◇ Spark Structured Streaming

- Consumes the streaming transactions from Kafka into Databricks.
- Provides a unified engine that supports both batch and streaming with minimal code changes (`read` → `readStream`).

◇ Auto Loader

- Incrementally ingests the fraud-watchlist JSON files as they land in cloud storage.
- Automatically tracks which files have been processed, infers schema, evolves it, and rescues corrupt data.

◇ Lakeflow Connect (Managed Ingestion)

- Ingests the customer master data from Neon PostgreSQL into Databricks.
- Enables no-code, incremental (CDC-style) batch ingestion using just a primary key and a cursor/timestamp column.

◇ Lakeflow Spark Declarative Pipelines (SDP)

- Orchestrates all Bronze/Silver/Gold transformations. (Formerly called Delta Live Tables / DLT.)
- Automatically manages checkpoints, table creation, dependencies, data-quality checks, and provides rich monitoring — reducing boilerplate code.

◇ Delta Lake

- The storage format for every Bronze, Silver, and Gold table.
- Provides ACID transactions, schema enforcement, and native support for streaming tables.

◇ Unity Catalog

- The governance layer where the `finguard` catalog and all schemas/tables live.
- Provides centralized security, lineage, and data governance out of the box.

◇ Gmail SMTP + Databricks Secret Scope

- Used to send real-time fraud alert emails to customers.
- The Secret Scope securely stores Kafka and Gmail credentials so they are never hard-coded.

◇ Lakeflow Jobs (Orchestration)

- Orchestrates the batch customer-ingestion pipeline (source → bronze → silver) on a daily schedule.
- The streaming pipeline runs continuously and needs no separate orchestration; the dashboard refreshes every minute.

---

4. Data Sources

FinGuard combines **three** different data sources — two streaming and one batch.

4.1 Confluent Kafka — Live Transactions (Streaming)

Credit card transactions are produced in real time to a Confluent Kafka topic named `credit_card_transactions` and consumed into Databricks.

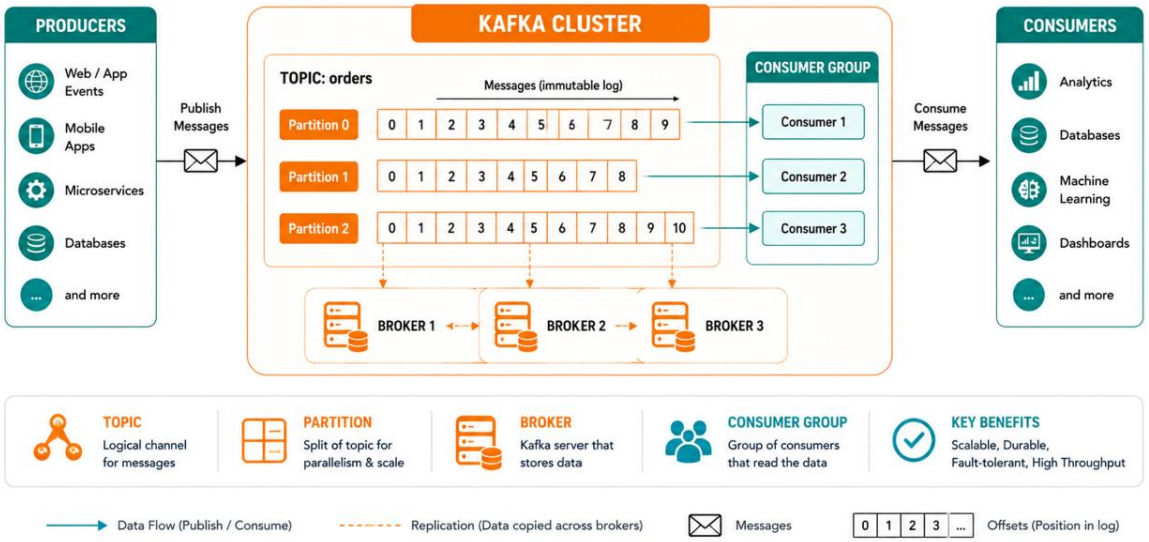


Figure 3: Kafka Architecture

Producers publish messages to a **topic**, which is split into **partitions** for parallelism and stored across **brokers** in the cluster; **consumers** (organized into **consumer groups**) then read those messages in parallel, tracking their position using **offsets**. Understanding these building blocks helps in relating to the pipeline:

- **Producer** → The data generator (web/mobile apps, microservices, databases) that publishes messages.
- **Topic** → A logical channel for messages (similar to a table in a database); messages are sent to and read from it.
- **Partition** → Sub-divisions within a topic that enable parallel writes and reads (this project uses **6 partitions**).
- **Offset** → The position of a message within a partition; consumers track offsets to know exactly what they have already read.
- **Broker** → The actual Kafka server/node that stores the data.
- **Cluster** → The collection of brokers where all producer messages are stored.
- **Consumer** → The entity that reads/consumes messages from a topic.
- **Consumer Group** → A set of consumers that share the work, each reading a subset of partitions in parallel.

4.2 JSON Files — Fraud Watchlist (Streaming)

- A generator continuously drops **JSON files** (one record per file, every ~5 seconds) into a Databricks **Volume**.
- Each file represents a card/entity flagged by internal fraud teams or external partners (e.g., a card blocked for phishing).
- Fields include `watchlist_id`, `watch_type`, `entity_id` (card number), `risk_level`, `action`, `reason_code`, `reason_description`, `status`, `effective_from`, `reported_by`, `reported_source`, `country`, and `city`.
- Ingested in real time using **Auto Loader**.

4.3 Neon PostgreSQL — Customer Master Data (Batch)

- A hosted PostgreSQL database (**Neon**) stores the **customer master data** (~1,000 customers).
- Contains reference attributes such as `customer_id`, name, gender, age, city/state/country, `annual_income`, `customer_segment`, `card_number`, `card_type`, `email`, and — most importantly — the per-customer `transaction_limit`.
- This data is relatively static (it doesn't change every second), so it is ingested as a **batch/incremental load** using **Lakeflow Connect** with `customer_id` as the primary key and `update_timestamp` as the cursor column.

---

5. Data Pipelines

All processing follows the **Medallion Architecture** — Bronze (raw), Silver (cleaned/standardized), and Gold (business-ready) — inside the `finguard` Unity Catalog.

5.i) Source → Bronze Ingestion

The Bronze layer captures raw data from each source with minimal transformation, adding tracking columns such as the ingestion timestamp.

- **Transactions (from Kafka):** Read using `spark.readStream.format("kafka")` with SASL_SSL/PLAIN authentication (credentials pulled from the Secret Scope). The raw `key` and `value` are cast to string and stored alongside Kafka metadata columns and an ingestion timestamp. The value column holds the full transaction JSON, still unparsed at this stage.

- **Fraud Watchlist (from JSON files):** Read using Auto Loader (`format("cloudFiles")`) with JSON format and automatic schema inference. A `_rescued_data` column (added automatically by Auto Loader) captures any corrupt data, and the source file path is tracked in `source_file`.
- **Customers (from PostgreSQL):** Ingested via **Lakeflow Connect** (managed ingestion pipeline), which handles historical + incremental loads with no custom code.

Bronze Tables

#	Bronze Table	Source	Ingestion Method
1	`finguard.bronze.transactions`	Confluent Kafka	Spark Structured Streaming
2	`finguard.bronze.fraud_watchlist`	JSON files (Volume)	Auto Loader
3	`finguard.bronze.customers`	Neon PostgreSQL	Lakeflow Connect (batch/incremental)

5.ii) Bronze → Silver Ingestion

The Silver layer parses, cleans, and standardizes the data, and applies **Data Quality (DQ) checks**. In Lakeflow SDP, DQ rules are declared with `@expect_or_drop` (fail → drop the record) or `@expect` (fail → flag/warn but keep the record).

Bronze → Silver Mapping & Transformations

Silver Table	Source (Bronze)	Key Transformations	Data Quality Checks
`finguard.silver.transactions`	`finguard.bronze.transactions`	Parse the Kafka `value` JSON into individual typed columns using a defined schema; rename Kafka metadata (`kafka_topic`, `kafka_partition`, `kafka_offset`, `kafka_timestamp`); add `bronze_ingestion_timestamp` `valid_transaction_id` and `silver_ingestion_timestamp`.	→ not null *(drop)* `valid_customer_id` → not null *(drop)* `valid_card_number` → not null *(drop)* `valid_merchant_id` → not null *(drop)* `valid_amount` → amount > 0 *(warn only)*
`finguard.silver.fraud_watchlist`	`finguard.bronze.fraud_watchlist`	Standardize key fields to **UPPERCASE** (`watchlist_id`, `entity_id`, `risk_level`, `action`); convert `effective_from` from string to **timestamp**; drop `_rescued_data`; rename ingestion timestamps to bronze/silver.	*(Cleaning & standardization; no drop rules)*
`finguard.silver.customers`	`finguard.bronze.customers`	Select and retain all customer attributes; convert `account_open_date` from string to **date**; add `silver_ingestion_timestamp`.	`valid_customer_id` → not null *(drop)*

Even though the customer data is loaded in batch, the Silver customer table is read as a streaming table so that incremental capabilities are handled automatically by Databricks.

5.iii) Silver → Gold Ingestion

The Gold layer produces business-ready tables for fraud alerting and analytics. Four Gold tables are built:

Gold Table	Purpose	How the Data Is Prepared
`finguard.gold.high_value_transactions_alert`	Detects transactions **exceeding the customer's configured limit**.	**Stream-static join** of Silver transactions (stream) with Silver customers (static), filtered where `amount > transaction_limit`; builds an alert record with `alert_id`, `alert_type = HIGH_VALUE_TRANSACTION`, customer email, amount, limit, merchant, location, etc.
`finguard.gold.fraud_card_alert`	Detects transactions on **cards present in the fraud watchlist**.	**Stream-stream join** (with watermarks) of Silver transactions and Silver fraud watchlist on `card_number = entity_id` (**inner join**), then a **left join** to customers to fetch the email; builds an alert with `alert_type = FRAUD_WATCHLIST_MATCH`, risk level, reason, and full transaction + watchlist details.
`finguard.gold.transaciton_count_by_minute`	Counts transactions per **1-minute tumbling window**.	Group Silver transactions by a **tumbling window** of 1 minute; output `window_start`, `window_end`, `transaction_count`.
`finguard.gold.transaciton_count_by_minute_sliding_window`	Counts transactions over a **rolling 5-minute window updated every minute**.	Group by a **sliding window** (size 5 min, slide 1 min); output `window_start`, `window_end`, `transaction_count`.

a. Stateless vs. Stateful Transformation

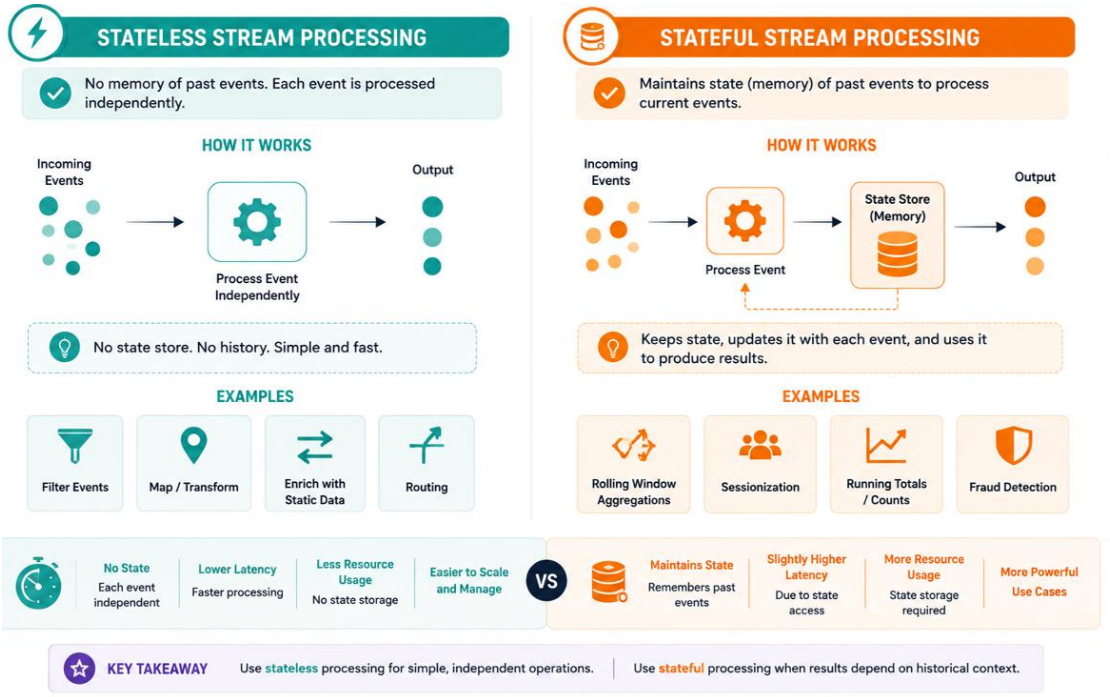


Figure 4: Stateless vs Stateful Stream Processing

Stateless processing handles each event independently and keeps no memory of past events, while **stateful** processing maintains information across events to support aggregations, windowing, sessionization, and advanced fraud detection. The table below summarizes the key differences:

Aspect	Stateless	Stateful
Definition	No memory of past events required; each event is processed independently.	Requires maintaining state/memory of past events to process the current event.
Examples	Simple filters, one-to-one maps/casts, enrichment with static data (stream-static join), simple routing.	Windowed/rolling aggregations, running totals, per-user sessions, advanced fraud detection (stream-stream joins).
Latency	Lower — each event processed as it arrives.	Higher — depends on how much memory/state is kept.
Resource Usage	Lower — single-record processing.	Higher — processes sets of records together.
Scalability	Easier to scale and manage.	Harder to manage, but worth it for powerful use cases.

In FinGuard, the transactions-only processing (parsing, casting, filtering, stream-static enrichment) is stateless, while the stream-stream join and the windowed aggregations are stateful.

b. Watermarks

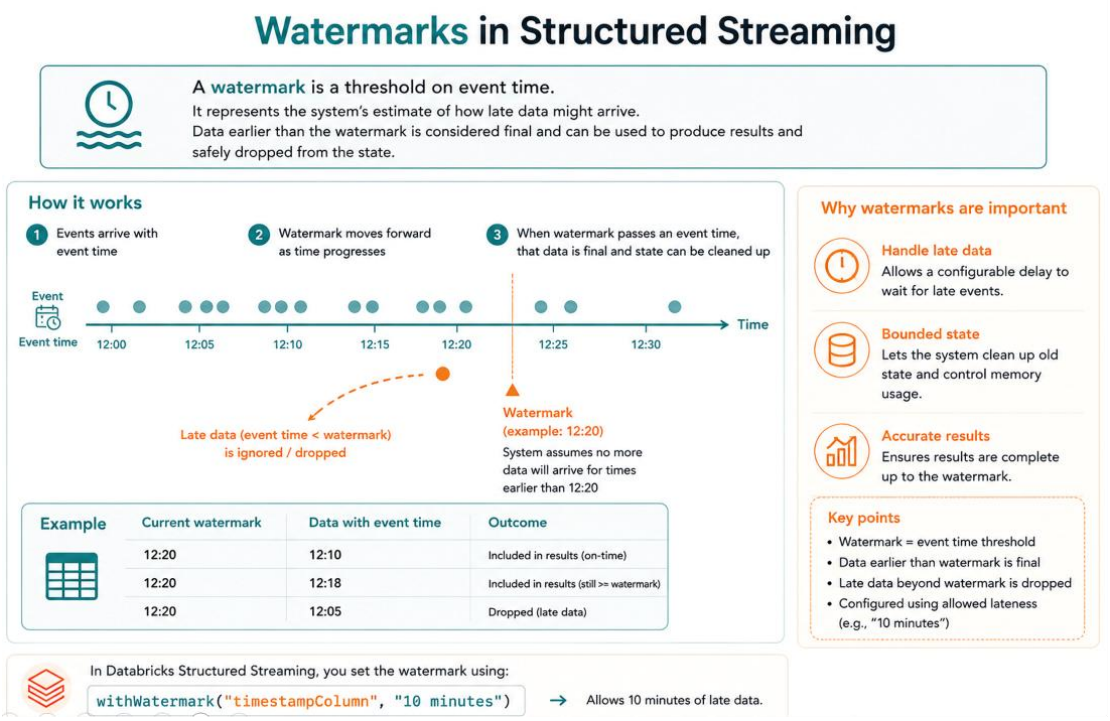


Figure 5: Watermarks in Structured Streaming

A watermark defines a threshold for accepting late data, which enables automatic state cleanup, improves memory efficiency, and balances processing latency against result accuracy. The key points are:

- A **watermark** is a threshold on the **event time** that defines how late data can arrive before it is ignored.
- It is set with `withWatermark("<event_time_column>", "<duration>")`; in FinGuard it is **5 minutes**.
- **Important:** the threshold is measured from the **latest event time seen** in the stream — *not* from the current wall-clock time. If the latest event is at 12:30 with a 10-minute watermark, anything older than 12:20 is dropped.
- Watermarks are essential for stream-stream joins and windowed aggregations because they **bound the state** kept in memory, handle late-arriving data, and produce accurate results.

c. Stream-Static Join

- Joins a **streaming** table with a **static/batch** table (transactions ⋈ customers).
- The static table is read with `spark.read.table(...)` (not `readStream`).
- Always uses a **left join** — every incoming transaction is checked against the customer reference data; a plain inner join is avoided because streaming data keeps arriving continuously.
- Used to build the **high-value transaction alert** by comparing each transaction's amount to the customer's limit.

d. Stream-Stream Join

- Joins **two streaming** tables (transactions ⋈ fraud watchlist).
- **Both** streams must define **watermarks** (5 minutes each here) so state is bounded.
- Uses an **inner join** on `card_number = entity_id` — a fraud is confirmed only when a transaction matches a watchlisted card within the watermark window.
- Used to build the **fraud card alert**; late data beyond the watermark is automatically ignored.

e. Windows — Tumbling & Sliding

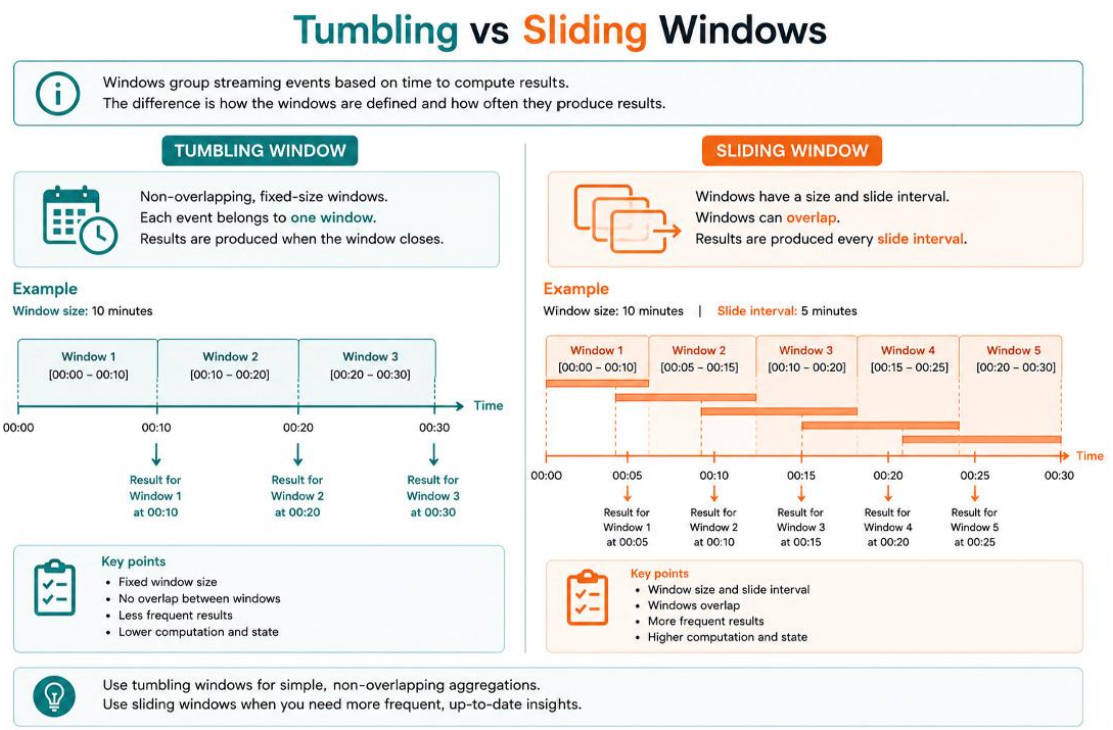


Figure 6: Tumbling vs Sliding Windows

Tumbling windows divide data into fixed, non-overlapping intervals, while **sliding** windows use overlapping intervals to generate more frequent results. The table below maps each to its FinGuard configuration:

Window Type	Description	FinGuard Configuration
Tumbling	Fixed-size, **non-overlapping** windows; each event belongs to exactly one window; results emitted when the window closes.	**1-minute** window → count of transactions per minute.
Sliding	Fixed-size windows that **overlap** and slide forward by a set interval; an event can belong to multiple windows; gives more frequent, up-to-date insights.	**5-minute** window, sliding every **1 minute** → rolling count of the last 5 minutes, refreshed each minute.

Both are used to "bound the unbounded stream" — providing a boundary so aggregations on continuously arriving data can produce results.

5.iv) Sending Real-Time Email Alerts

Whenever a Gold alert table receives a new record, a real-time email is sent to the affected customer. This is implemented in the SDP pipeline using a `@foreach_batch_sink` (which processes each streaming micro-batch) combined with an `@append_flow` (which continuously feeds new alert rows from the Gold table into the sink). Emails are sent via the **Gmail SMTP server**, with the Gmail app password securely retrieved from the **Databricks Secret Scope**. Two alerts are configured:

Alert 1 — High-Value Transaction Alert

- **Reads from:** `finguard.gold.high_value_transactions_alert`
- **Triggers when:** a transaction amount exceeds the customer's configured `transaction_limit`.
- **Email content:** A formatted HTML email showing the transaction ID, amount vs. the customer's limit, merchant, location, and date/time, plus guidance on what to do if the transaction was not authorized.

Alert 2 — Fraud Card Alert

- **Reads from:** `finguard.gold.fraud_card_alert`
- **Triggers when:** a transaction is made on a card that matches the fraud watchlist.
- **Email content:** A more detailed, high-urgency HTML email including the risk level, watchlist type, reason, masked card number (last 4 digits only), full transaction details, watchlist information, and immediate action steps (e.g., call the fraud hotline).

📌 Note: Because these emails are sent from a personal Gmail account and mention fraud/transactions, they may initially land in the recipient's Spam folder.

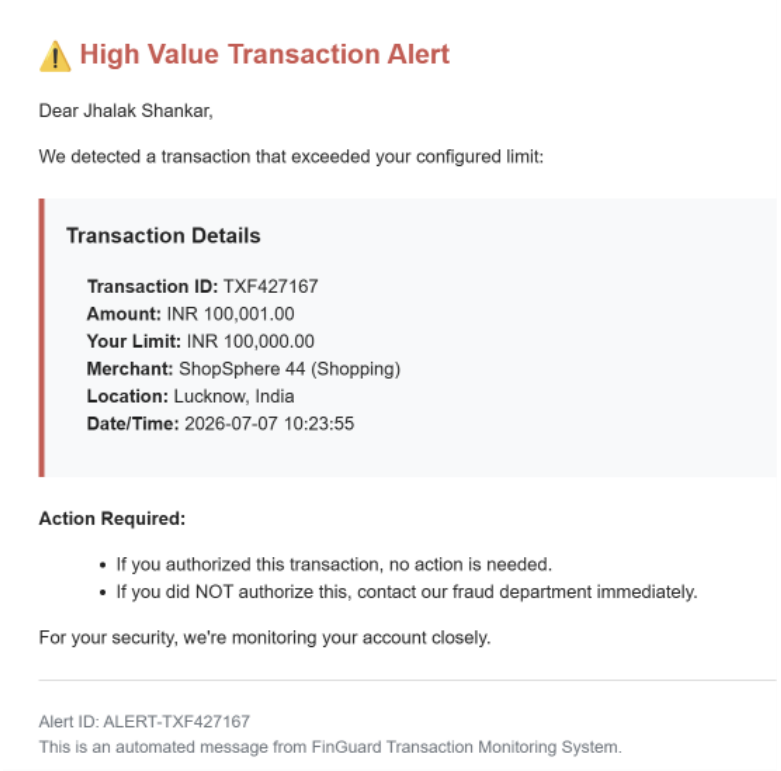


Figure 7: Sample Email

---

6. Dashboard

A Databricks dashboard — **FinGuard Fraud Detection Monitoring** (page: *Fraud Monitoring*) — provides near-real-time operational visibility. It is built on the Silver and Gold tables and is configured to **refresh every 1 minute** for minute-level monitoring.

📊 Key KPIs (Counters)

- **Total Alerts Today**
- **Total Transactions Today**
- **Average Transaction Amount**
- **High Risk Customers Count**

Charts & Visualizations

- **Alert Trends Over Time** (line) — alerts grouped by hour over the last 24 hours.
- **Top 10 Customers by Alert Count** (bar).
- **Alert Type Breakdown** (pie) — high-value vs. fraud-watchlist alerts.
- **Alert Status Distribution** (pie).
- **Transaction Volume by Minute** (line) — from the windowed aggregation table.
- **Transaction Amount by Merchant Category** (bar).
- **International vs. Domestic Transactions** (pie).
- **Top 10 Merchants by Transaction Count** (bar).
- **Transaction Heatmap by Country** (bar).
- **Payment Channel Distribution** (pie).
- **Risk Level Distribution from Watchlist**.

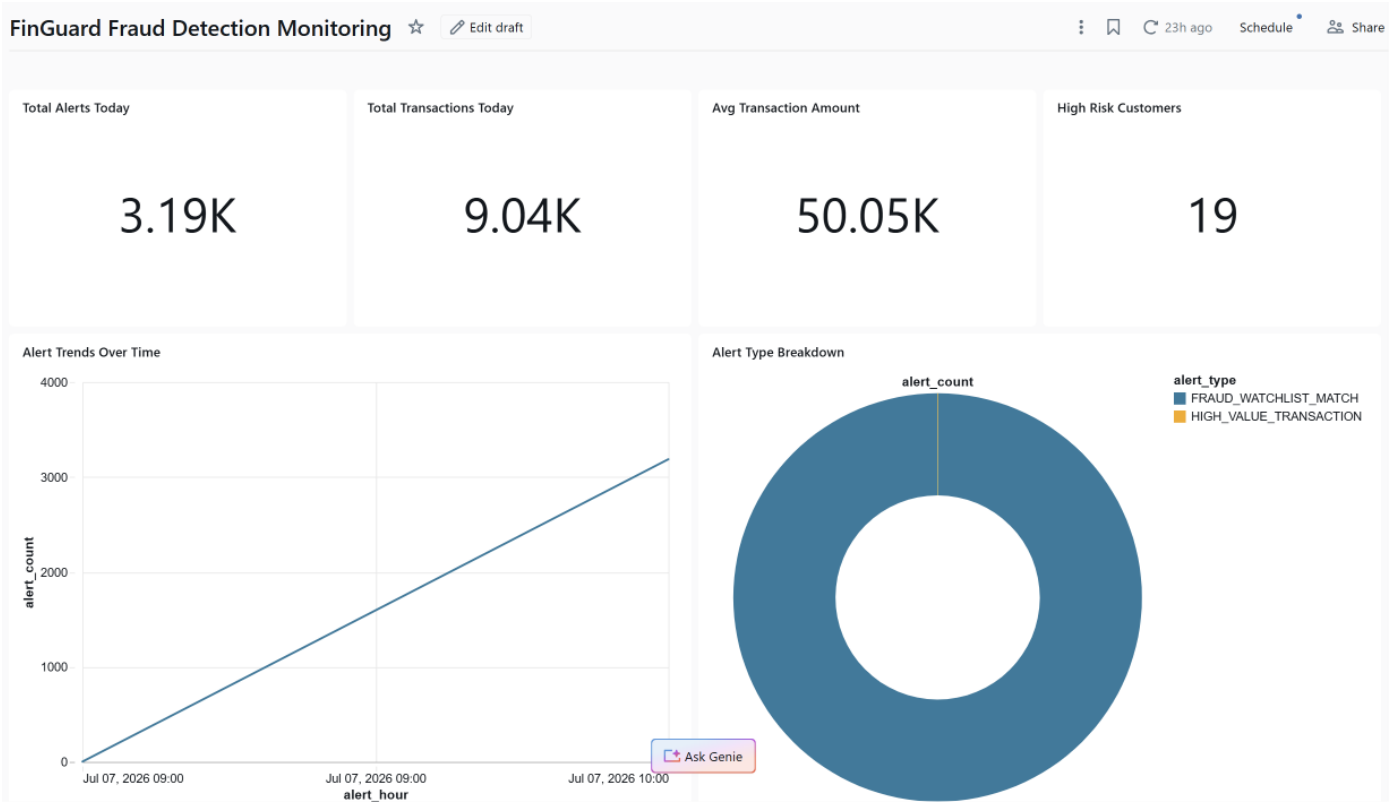


Figure 7: Dashboard Screenshot

---

7. Project Details for Resume

Project Overview & Duration

FinGuard — Real-Time Credit Card Fraud Detection Platform | *Databricks | Spark Structured Streaming | Lakeflow | 4 months*

Designed and built an end-to-end real-time fraud detection platform for a banking use case that ingests live credit card transactions from Confluent Kafka, fraud watchlist JSON files, and customer master data from PostgreSQL, then detects fraudulent and high-value transactions within seconds and triggers real-time email alerts and a live monitoring dashboard. Implemented on Databricks using a Bronze–Silver–Gold medallion architecture on Delta Lake with Unity Catalog governance.

Skills Used

- **Streaming & Processing:** Apache Spark Structured Streaming, Spark Declarative Pipelines (Lakeflow / Delta Live Tables), Auto Loader
- **Messaging / Ingestion:** Confluent Apache Kafka (producers, topics, partitions, offsets, consumer groups), Lakeflow Connect (managed / CDC ingestion)
- **Streaming Concepts:** Stateless vs. stateful processing, watermarking, stream-static joins, stream-stream joins, tumbling & sliding window aggregations, checkpointing, triggers (Available Now / continuous)
- **Data Platform & Storage:** Databricks, Delta Lake, Unity Catalog, Medallion Architecture, Databricks Volumes
- **Data Sources:** Confluent Kafka, JSON files, Neon PostgreSQL
- **Data Quality:** Declarative expectations (`expect` / `expect_or_drop`)
- **Programming & Tooling:** Python, PySpark, SQL, Databricks Secret Scopes, Gmail SMTP integration
- **Orchestration & BI:** Lakeflow Jobs, Databricks Dashboards (near real-time)

Roles & Responsibilities

1. Ingested streaming transactions from Kafka into the **Bronze** layer using **Spark Structured Streaming**, and fraud-watchlist JSON files using **Auto Loader**.
2. Ingested **customer master data** from **Neon PostgreSQL** into Databricks using **Lakeflow Connect** with incremental (CDC) loading.
3. Built the **Silver** layer with schema parsing, data cleansing, timestamp/date conversions, standardization, and declarative **data-quality checks**.
4. Engineered the **Gold** layer to detect fraud in real time using **stream-static joins** (high-value transactions), **stream-stream joins with watermarks** (fraud watchlist matches), and **tumbling & sliding window aggregations**.
5. Implemented **real-time email alerting** via `foreach_batch_sink` + `append_flow` using Gmail SMTP, with credentials secured in **Databricks Secret Scopes**.
6. Built a **near-real-time monitoring dashboard** (KPIs, alert trends, transaction analytics) refreshing every minute.
7. **Orchestrated** the solution end to end using **Lakeflow Jobs** (daily batch for customer data, continuous streaming pipeline, and 1-minute dashboard refresh), with all assets governed under **Unity Catalog**.